Neural Network Day 4

학습 기법 (training techniques)

HYUNDAI AI Insight Campus · 온디바이스 AI · Neural Network 4/5

2026. 10. 23 (금) 09:00–18:00 · 8교시 · Google Colab

Optimizer | Xavier · He | Batch Norm | Overfitting | Weight decay · Dropout | mynn.py

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 Optimizer

SGD 가 지그재그하는 이유를 설명하고 Momentum · AdaGrad · RMSProp · Adam 을 구현한다

2 가중치 초기화

활성화 분포를 그려 보고 sigmoid 에는 Xavier, ReLU 에는 He 를 고른다

3 Batch Normalization

BatchNorm forward / backward 를 구현하고 gradient check 로 검증한다

4 Overfitting · 규제

train / test 간극을 측정하고 weight decay · Dropout 으로 줄인다

5 Hyperparameter 탐색

validation 을 분리하고 로그 스케일 랜덤 탐색으로 lr · λ 를 고른다

6 미니 프레임워크

mynn.py (Sequential · Trainer) 로 5 층 MLP 를 학습해 test ≥ 0.97 을 낸다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 최적화 · 초기화 , 오후 BN → 과적합 → 규제 → 탐색 → 프레임워크

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 3 복습 퀴즈 · SGD 의 약점	퀴즈 + 강의	D4-1 SGD 지그재그
2	10:00–10:50	Optimizer — Momentum · AdaGrad · RMSProp · Adam	강의 + 실습	D4-2 옵티마이저 경주
3	11:00–11:50	가중치 초기화 — 대칭 문제 · Xavier · He	강의 + 실습	D4-3 초기화 · 활성화 분포
4	13:00–13:50	Batch Normalization — forward / backward	강의 + 실습	D4-4 Batch Normalization
5	14:00–14:50	Overfitting — train / test 간극	강의 + 실습	D4-5 과적합
6	15:00–15:50	규제 — weight decay · Dropout	강의 + 실습	D4-6 weight decay · Dropout
7	16:00–16:50	Hyperparameter 탐색 — validation · 랜덤 탐색	강의 + 실습	D4-7 하이퍼파라미터 탐색
8	17:00–17:50	미니 프로젝트 — mynn.py 미니 프레임워크 (test ≥ 0.97)	개인 실습	D4-8 미니 프레임워크

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 검증 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 4)

https://imguru-mooc.github.io/AI_CAMPUS/NN-day4/

animation.html	애니메이션 8 개 (D4-1 ~ D4-8) — #d4-4 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기
NN_Day4_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + mynn.py 미니 프로젝트 (Colab 에서 열기)
NN_Day4.pdf	오늘 슬라이드 PDF
Day 3 복습 퀴즈	09:00 시작 — 강사가 Google Forms 링크 공유 (20 문항 · 15 분)

노트북은 " 파일 → Drive 에 사본 저장 " 후 작업 · 8 교시 mynn.py 는 Drive 에 저장해 Day 5 에서 import 합니다

오늘의 큰 그림 — 같은 MNIST, 더 잘 학습시키기

Day 3 에서 " 학습이 된다 " 를 확인했습니다 . 오늘은 같은 데이터로 더 빠르게 , 더 안정적으로 , 덜 외우게

Day 3 (어제)

```
784-100-10
std 0.01 · SGD lr 0.1
10 epoch
```

test 0.9694

오늘 더하는 것

더 빠르게	Optimizer (Adam)
더 안정적으로	He 초기화 · Batch Norm
덜 외우게	weight decay · Dropout

Day 4 (오늘)

```
784-100×4-10 + BN
He · Adam lr 0.001
5 epoch
```

test 0.9757

오늘의 흐름 — Day 3 끝에 남긴 숙제에서 출발

SGD 지그재그 (1–2 교시) → 초기화 · BN (3–4 교시) → 과적합 · 규제 (5–6 교시) → 탐색 (7 교시) → mynn.py (8 교시)

" 나선 손실 곡선이 흔들렸고 MNIST 도 10 epoch" → 오늘 끝에는 5 epoch 에 0.9757, 그리고 Day 5 CNN 이 쓸 mynn.py

1 Day 3 복습 퀴즈 · SGD 의 약점

09:00–09:50

학습 목표

- 09:00–09:15 Day 3 복습 퀴즈 (20 문항)
- 비등방성 함수 : y 방향이 20 배 가파름
- lr 크면 y 지그재그, 작으면 x 가 느림
- y 방향 안정 조건 : lr < 1

노트북 실습

1. 문제 1-1 grad_f
2. 문제 1-2 SGD 경로 (run_optimizer)
3. 문제 1-3 학습률 비교

▶ 애니메이션 D4-1 SGD 지그재그

핵심 개념

함수

```
f(x, y) = x²/20 + y²
grad = (x/10, 2y),   시작 (−7, 2), 30 step
```

SGD lr 0.95

```
y ← y − 0.95·2y = −0.9·y  (매 step 부호가 바뀜)
끝 (−0.350, 0.085) — 지그재그
```

lr 비교

```
lr 0.1  → (−5.178, 0.002)  x 가 느림
lr 1.0  → y 가 ±2 에서 영원히 진동
lr 1.05 → 발산 (y = 34.9)
```

안정 조건

```
y ← (1 − 2·lr)·y → |1 − 2·lr| < 1 → lr < 1
```

체크포인트

x 는 완만해서 큰 lr 이 필요하고 y 는 가팔라서 lr < 1 이어야 한다 — 학습률 하나로 두 방향을 다 만족시킬 수 없다.

2 Optimizer 구현

10:00–10:50

학습 목표

- 모두 update(params, grads) — 제자리 갱신
- 상태 (v, h, m) 는 첫 update 때 np.zeros_like
- Momentum = 속도 누적 , AdaGrad · RMSProp = 축마다 lr 조절
- Adam = Momentum + RMSProp + 편향 보정

노트북 실습

1. 문제 2-1 Momentum
2. 문제 2-2 AdaGrad
3. 문제 2-3 RMSProp
4. 문제 2-4 Adam
5. 문제 2-5 경로 비교
6. 문제 2-6 MNIST 비교

▶ 애니메이션 D4-2 옵티마이저 경주

핵심 개념

Momentum

$$v \leftarrow \alpha v - \eta \cdot \partial L/\partial W \quad (\alpha = 0.9)$$
$$W \leftarrow W + v$$

AdaGrad RMSProp

$$\text{AdaGrad: } h \leftarrow h + g^2$$
$$\text{RMSProp: } h \leftarrow \rho h + (1-\rho) g^2 \quad (\rho = 0.99)$$
$$W \leftarrow W - \eta \cdot g / (\sqrt{h} + 1e\text{-}7)$$

Adam

$$m \leftarrow \beta_1 \cdot m + (1-\beta_1) g, \quad v \leftarrow \beta_2 \cdot v + (1-\beta_2) g^2$$
$$\hat{m} = m/(1-\beta_1^t), \quad \hat{v} = v/(1-\beta_2^t)$$
$$W \leftarrow W - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + 1e\text{-}7) \quad (\beta_1\ 0.9,\ \beta_2\ 0.999)$$

30 step 끝 f 값

AdaGrad 0.00072 · SGD 0.0133 · RMSProp 0.0243
Momentum 0.0479 · Adam 0.102
f 가 작다 ≠ 좋은 optimizer — 경로 모양을 본다

체크포인트

MNIST 784-100×4-10 · He · batch 128 · 2000 iter → test SGD(0.01) 0.9378 · Momentum(0.01) 0.9644 · AdaGrad(0.01) 0.9726 · RMSProp(0.001) 0.9745 · Adam(0.001) 0.9740

3 가중치 초기화

11:00–11:50

학습 목표

- W 를 모두 같은 값으로 두면 대칭이 깨지지 않는다
- std 가 크면 sigmoid 포화 → 기울기 소실
- std 가 작으면 활성값이 한 점에 몰림
- sigmoid · tanh 는 Xavier, ReLU 는 He

노트북 실습

1. 문제 3-1 init_scale (Xavier · He)
2. 문제 3-2 활성화 히스토그램
3. 문제 3-3 MNIST 초기화 비교

▶ 애니메이션 D4-3 가중치 초기화 · 활성화 분포

핵심 개념

대칭 문제

```
784-10-10, W 전부 0.01, SGD 0.1, 300 iter
→ W1 의 모든 열이 똑같음, test 0.2714
```

init_scale

```
Xavier: std = √(1/n_in)   (sigmoid · tanh)
He:     std = √(2/n_in)   (ReLU)
```

ReLU 5 층 활성값 std

```
std 0.01: 0.058 → 0.0041 → 0.0003 → 0.0000
Xavier:   0.583 → 0.412 → 0.303 → 0.220 → 0.177
He:       0.82 → 0.82 → 0.86 → 0.88 → 1.00
```

MNIST 2000 iter

```
std 0.01 → 손실 2.301 그대로, test 0.1135
Xavier → 0.9179   ·   He → 0.9378
```

체크포인트

sigmoid std 1 → 0 · 1 근처 비율 0.82/0.75/0.72/0.74/0.76 (기울기 소실), std 0.01 → 모두 0.5 근처 (표현력 제한), Xavier 는 고르게 퍼진다 .

4 Batch Normalization

13:00–13:50

학습 목표

- 미니배치마다 평균 0 · 분산 1 로 정규화
- γ, β 로 다시 늘이고 옮긴다 (학습 파라미터)
- 학습 땐 배치 통계 , 추론 땐 running 값
- 초기화에 덜 민감 , 학습이 빨라진다

노트북 실습

1. 문제 4-1 BatchNorm.forward
2. 문제 4-2 BatchNorm.backward
3. 문제 4-3 gradient check
4. 문제 4-4 BN 유무 비교

▶ 애니메이션 D4-4 Batch Normalization

핵심 개념

forward

```
mu = x.mean(axis=0); var = x.var(axis=0)
xhat = (x − mu) / np.sqrt(var + 1e-7)
out = gamma * xhat + beta
```

backward

```
dxhat = gamma * dout
dgamma = Σ(xhat·dout),  dbeta = Σdout
dx = (N·dxhat − Σdxhat − xhat·Σ(dxhat·xhat)) / (N·std)
```

검증

```
gradient check: dx 3.8e-10 · dγ 9.1e-13 · dβ 6.8e-14
dx 의 열 합 = 0 → BN 앞 Affine 의 db = 0
```

BN 효과 std 0.01

```
1000 장 · 784-100×5-10 · SGD 0.01 · 20 epoch
noBN train/test 0.117/0.103 → BN 1.000/0.878
```

체크포인트

running_mean = 0.9·running_mean + 0.1·μ — N(5, 2²) 배치 100 개를 50 번 넣으면 5.03 / 3.83. 추론은 train_flg=False.

5 Overfitting (과적합)

14:00–14:50

학습 목표

- train 은 오르는데 test 는 멈춘다
- test 손실은 오히려 증가 (틀린 답을 과신)
- 원인 : 데이터는 적고 모델은 크다
- 처방 1 순위 : 데이터를 늘린다

노트북 실습

1. 문제 5-1 train/test 간극 계산
2. 문제 5-2 데이터 크기 실험
3. 문제 5-3 곡선 그리기

▶ 애니메이션 D4-5 과적합

핵심 개념

실험

```
학습 300 장 · 784-100×6-10 (He) · SGD 0.1
batch 100 · 200 epoch · 10 epoch 마다 평가
```

결과

```
train 1.000 / test 0.786 → 간극 0.214
test 손실 25 epoch 0.79 → 200 epoch 1.12 ( 증가 )
```

데이터 ↑

```
3000 장 ( 같은 600 iteration = 20 epoch)
→ 1.000 / 0.926
```

모델 ↓

```
784-100-10 (300 장 , 200 epoch)
→ 1.000 / 0.802
```

체크포인트

정확도만 보면 놓친다 — test 손실이 다시 오르기 시작하면 과적합 . 데이터를 못 늘릴 때는 6 교시 규제로 .

6 규제 — weight decay · Dropout

15:00–15:50

학습 목표

- weight decay (L2): 큰 가중치에 벌점
- 편향 · γ · β 는 벌점에서 제외
- Dropout: 학습 때 뉴런 일부를 무작위로 끔
- 추론 때는 모두 켜고 (1 – ratio) 배

노트북 실습

1. 문제 6-1 l2_penalty
2. 문제 6-2 Dropout
3. 문제 6-3 세 가지 비교

▶ 애니메이션 D4-6 규제 — weight decay · Dropout

핵심 개념

weight decay

```
loss += 0.5 * lam * Σ np.sum(W**2)
dW   += lam * W
```

Dropout

```
train: mask = rng.random(x.shape) > ratio
       out = x * mask;  backward: dout * mask
추론 :  out = x * (1 – ratio)
```

결과 (5 교시 설정)

```
규제 없음     1.000 / 0.786
λ 0.01        1.000 / 0.810  test 손실 0.74 → 0.65
Dropout 0.2   1.000 / 0.800  (25 epoch train 0.53)
```

너무 강하면

```
λ 0.1 → 0.767 / 0.641 ( 과소적합 )
```

체크포인트

규제는 train 점수를 조금 내주고 test 를 올린다 . λ · ratio 도 직접 골라야 하는 값 → 7 교시 hyperparameter 탐색 .

7 Hyperparameter 탐색

16:00–16:50

학습 목표

- train / validation / test 분리
- test 로 고르면 test 에 과적합된다
- 로그 스케일 랜덤 탐색 > 그리드 탐색
- 넓게 → 좁게 범위를 줄여 가며 반복

노트북 실습

1. 문제 7-1 validation 분리
2. 문제 7-2 로그 스케일 샘플링
3. 문제 7-3 탐색 루프 · 산점도

▶ 애니메이션 D4-7 하이퍼파라미터 탐색

핵심 개념

validation

```
rng = default_rng(0); p = rng.permutation(1000)
val = p[:200];  train = p[200:]
```

샘플링 (30 회)

```
lr  = 10 ** rng.uniform(-4, 0)
lam = 10 ** rng.uniform(-6, -1)
784-100-100-10 · SGD · 20 epoch · batch 100
```

상위 3

```
val 0.885  lr 0.242  λ 4.8e-6
val 0.875  lr 0.240  λ 4.1e-5
val 0.870  lr 0.588  λ 1.0e-2
```

결론

```
lr 0.1 ~ 0.6 이 좋고 λ 는 영향이 작다
→ 다음 : lr = 10 ** U(−1.3, −0.2) 로 좁혀 다시
```

체크포인트

하위는 lr 3e-4 ~ 6e-4 근처 val 0.08 ~ 0.14 (학습이 안 됨). 30 회 탐색 약 11 초 — 고르는 데는 validation, 보고는 test 한 번 .

8 미니 프로젝트 — mynn.py 미니 프레임워크

17:00–17:50

강의 노트북 8 교시 · %%writefile → import mynn

```
계층 : Affine · Relu · Sigmoid · BatchNorm · Dropout
옵티마이저 5 종 : SGD · Momentum · AdaGrad · RMSProp · Adam
Sequential(layers, weight_decay_lambda)   # 키 "i.W"
Trainer(model, optimizer, batch_size, seed).fit(...)
save_params / load_params
```

과제	내용	배점
1	Sequential — params · grads · loss · gradient	20
2	Trainer.fit — 셔플 · 미니배치 · history	20
3	5 층 MLP 구성 · 학습 — test ≥ 0.97	30
4	저장 · 다시 불러와 같은 정확도	20
5	결과 해석 (강사 채점)	10

채점 셀 실행 → 자동 채점 90 점 + 해석 10 점

▶ D4-8 에서 계층 조립 · 학습 흐름을 먼저 보고 코드로 확인

기준 모델 — 오늘 배운 것이 모두 들어간다

```
Affine(784, 100, seed=1) → BatchNorm(100) → Relu
Affine(100, 100) → BN → Relu   × 3 (seed 2~4)
Affine(100, 10, seed=5) → SoftmaxWithLoss
```

파라미터 **110,610** (BN γ · β 800 포함) · 학습 전 손실 2.49

```
Trainer(model, Adam(lr=0.001), batch_size=100, seed=0)
```

epoch	loss	train	val (= test)
1	0.2917	0.9731	0.9636
2	0.1062	0.9847	0.9709
3	0.0751	0.9864	0.9734
4	0.0577	0.9912	0.9750
5	**0.0474**	**0.9920**	**0.9757**

목표 · 참고 결과 (5 epoch)

목표 test ≥ 0.97 · 기준 5 epoch 0.9757 (서버 약 15 초)
Day 3 0.9694 (10 epoch) → Day 4 0.9757 (5 epoch)

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

SGD 의 약점	가파른 방향은 지그재그 , 완만한 방향은 느림 — y 방향 안정 조건 lr < 1	D4-1
Optimizer	Momentum · AdaGrad · RMSProp · Adam — 모두 update(params, grads) 제자리 갱신	D4-2
초기화	같은 값이면 대칭 , sigmoid · tanh 는 Xavier √(1/n), ReLU 는 He √(2/n)	D4-3
Batch Norm	배치 평균 0 · 분산 1 → γ, β · 추론은 running 값 (std 0.01 에서도 학습)	D4-4
Overfitting	train 1.000 / test 0.786 — test 손실 증가가 신호 , 데이터 ↑ 가 1 순위	D4-5
규제	weight decay ½λΣW² (+λW) · Dropout (추론 × (1 − ratio)) → test 0.810 / 0.800	D4-6
Hyperparameter	validation 으로 고르고 , 로그 스케일 랜덤 탐색 → 범위를 좁혀 반복	D4-7
mynn.py	Sequential · Trainer → 5 층 + He + BN + Adam, 5 epoch test 0.9757	D4-8

Day 5 로 가는 다리 — CNN

오늘 만든 mynn.py 를 import 해서, 이미지의 모양을 살리는 합성곱 신경망을 학습시킵니다

Day 4 (오늘)		Day 5 (다음 시간)
`Affine: 784 픽셀을 한 줄로 펼침`	→	Convolution: 28×28 모양을 유지
`Y = XW + b (모든 입력 × 모든 가중치)`	→	필터 · 패딩 · 스트라이드 — 출력 크기 계산
`for 루프로 짜면 느린 합성곱`	→	im2col 로 행렬 곱 하나로
`Affine · Relu forward / backward`	→	Convolution · Pooling forward / backward
`mynn.Sequential · Trainer`	→	SimpleConvNet 을 import mynn 으로 MNIST 학습
`count_params() = 110,610`	→	모델 비용 계산 (파라미터 · 연산량)

합성곱 · 패딩 · 스트라이드 → im2col → Convolution · Pooling → SimpleConvNet (import mynn) → 모델 비용

제출 : 강의 노트북 (mynn 채점 셀 결과 포함) · mynn.py 와 가중치를 Drive 에 저장 (Day 5 에서 import)